

מבוא לתכנות מערכות: חזרה למבחן

שאלה 1

ממשו פונקציה בשם `find_unique` שמקבלת שם של קובץ שמכיל בכל שורה מספר אחד שלם, ומדפיסה ל-`standard output` את המספרים שנקראו בלי חזרות. מספר השורות לא ידוע מראש. הניחו שהקובץ תקין – אין צורך לבדוק שהוא נפתח וגם אפשר להניח שבכל שורה יש מספר שלם. במידה ואתם מקצים זכרון בצורה דינמית אין צורך לבדוק את הצלחת ההקצאה. לצורך מימוש הפונקציה מותר לקרוא את הקובץ פעם אחת אבל אין צורך לדאוג שהתוכנית תהיה יעילה. למשל: עבור קובץ שמכיל את הטקסט הבא:

```
17
2
698
17
2
3
```

הפלט יהיה (לא בהכרח בפורמט או בסדר הזה)

```
17
2
698
3
```

שאלה 2

א. כתבו פונקציה בשם `reverse_fours` שמקבלת מצביע למערך של `unsigned char` ומחליפה כל קבוצה של ארבעה ביטים במערך עם הקבוצה שנמצאת באותו המרחק מהסוף. הסידור של כל קבוצה של 4 ביטים יישאר כמו הסידור המקורי. אין להקצות מערך נוסף (בצורה דינמית או לא דינמית) לצורך מימוש הפונקציה.

למשל: עבור מערך באורך 3 שמכיל את הביטים

```
10011100 11110011 00001010
```

הפונקציה תסדר את הביטים כך:

```
10100000 00111111 11001001
```

חתימת הפונקציה:

```
void reverse_fours(unsigned char* arr, unsigned int n)
```

שאלה 3

בשאלה זו אתם מתבקשים לממש באופן חלקי מילון (Dictionary) אשר שומר מילים ואת הפירוש שלהן. המילון לא ישמש לשמירת אף טיפוס אחר מלבד מחרוזות. המילון ימומש בעזרת המודול `Array` אשר ה-`header` שלו נתון בהמשך. כל מילה והפירוש שלה ישמרו ב-`struct` אותו תגדירו ושלא יהיה זמין למשתמש. המימוש יתבסס על `Array` אחד למילים ו-`Array` נוסף לפירוש שלהן, כך שהפירוש של מילה נמצאת באינדקס מסוים ב-`Array` המילים יהיה באותו האינדקס ב-`Array` הפירושים.

הניחו שכל המילים שמוכנסות למילון שונות זו מזו. במימוש של פונקציות אין צורך לבדוק שהקצאות זכרון הצליחו.

כתבו ממשק (קובץ h) למבנה הנתונים Dictionary כמתואר אשר מאפשר את הפעולות הבאות:

- יצירת Dictionary
- הריסת Dictionary
- הוספת מילה חדשה והפירוש שלה למילון
- מציאת פירוש עבור מילה כלשהי במילון. הפונקציה תחזיר את הפירוש, או NULL במידה והמילה לא קיימת במילון.
- החזרת כל המילים שמתחילות ברצף אותיות מסויים אותו מעביר המשתמש. למשל: אם המשתמש העביר "th" תחזיר הפונקציה מערך עם כל המילים שמתחילות ב-th. הפונקציה תקבל כתובת של מערך שהוקצה בצורה דינמית מהמשתמש ותשנה את גודלו בהתאם למספר המילים. הערך המוחזר של הפונקציה יהיה מספר המילים שנמצא
- החזרת מספר המילים במילון

```
#ifndef ARRAY_H
#define ARRAY_H

typedef struct Array* Array;
typedef void* Element;

// Create an empty array with no cells
Array ArrCreate(Element (*cpy)(Element), void (*fre)(Element));

// destroy an array
void ArrDestroy(Array arr);

// Get Array size
unsigned int ArrSize(Array arr);

// Change the size of the Array
void ArrResize(Array arr, unsigned int new_size);

// Insert an element to position pos in arr
Element ArrPut(Array arr, unsigned int pos, Element e);

// Get the element in position pos but leave it in the Array
Element ArrAt(Array arr, unsigned int pos);

// Get the element in position pos and remove it from the Array
Element ArrGet(Array arr, unsigned int pos);

#endif // ARRAY_H
```

ב. הגדירו את ה-struct הדרוש לצורך מימוש המילון וכן כל פונקציה הנדרשת לצורך השימוש ב-Array. ממשו את פונקציית יצירת המילון.

ג. ממשו את פונקציית הוספת מילה ופירושה למילון. במידה והמילה קיימת במילון הפונקציה לא תעשה דבר.